

ALY 6040 Data Mining Applications

Module 5 – Technique Practice

Shraddha Gupte



CPS - MPSA, Northeastern University

Prof. Arasu Narayan

October 21, 2024

1. Introduction

The excerpt is from Dr. Martin Luther King Jr.'s iconic "I Have a Dream" speech, delivered during the March on Washington for Jobs and Freedom on August 28, 1963. This speech stands as a powerful testament to the struggle for civil rights and equality in America. Dr. King articulates his vision of a future where people are judged not by their skin color but by the content of their character. Through a series of poignant dreams, he calls for an end to racial injustice and envisions a nation united in brotherhood and freedom. His message resonates across generations, inspiring movements for equality and justice worldwide, making it a cornerstone of American history and civil rights advocacy.

2. Problem Statement and Objective

The objective of this analysis is to examine the linguistic patterns and thematic emphasis in Martin Luther King Jr.'s "I Have a Dream" speech through text mining and word frequency analysis. By identifying key themes such as freedom, unity, and action, the analysis aims to provide actionable insights for educators, advocates, and policymakers to enhance communication strategies around civil rights, diversity, equity, and inclusion. This will help align future messaging with King's powerful rhetoric, ensuring that the speech's core values remain impactful in modern discourse and educational efforts.

3. Exploratory Data Analysis

a. Installing the libraries

```
# Installing necessary libraries

import nltk
!pip install wordcloud
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
import requests
from nltk.corpus import stopwords
import re
Requirement already satisfied: wordcloud in /opt/anaconda3/lib/python3.12/site-packages (1.9.3)
Requirement already satisfied: numpy>=1.6.1 in /opt/anaconda3/lib/python3.12/site-packages (from wordcloud) (1.26.4)
Requirement already satisfied: pillow in /opt/anaconda3/lib/python3.12/site-packages (from wordcloud) (10.3.0)
```

```
Requirement already satisfied: matplotlib in /opt/anaconda3/lib/python3.12/site-packages (from wordcloud) (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (1.2.0)
Requirement already satisfied: cyclor>=0.10 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (23.2)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.16.0)
```

Description:

This code installs and imports the necessary libraries for text processing and visualization. It installs wordcloud, a library used to generate word clouds that visually represent the frequency of words in a text. It also imports nltk, a natural language processing toolkit, and matplotlib for data visualization. The CountVectorizer from sklearn is used to convert text into a matrix of token counts. The requests library retrieves online text, and stopwords from nltk helps remove common, insignificant words. The re library is used for regular expressions to clean the text.

b. Loading Text Data

```
# Downloading NLTK stopwords
nltk.download('stopwords')

# Load text data from URL
url = "http://www.sthda.com/sthda/RDoc/example-files/martin-luther-king-i-have-a-dream-speech.txt"
response = requests.get(url)
text = response.text

[nltk_data] Downloading package stopwords to
[nltk_data] /Users/shraddha/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Description:

This code fetches the text of Martin Luther King Jr.'s "I Have a Dream" speech from a URL and stores it in the variable text. The text is stored as a raw string.

c. Cleaning the Text

```
# Define a function to clean the text
def clean_text(text):
    # Replace specific characters with space
    text = re.sub(r'[/@|]', ' ', text)
    # Convert to lower case
    text = text.lower()
    # Remove numbers
    text = re.sub(r'\d+', '', text)
    # Remove punctuation
    text = re.sub(r'[^\w\s]', '', text)
    # Remove extra whitespaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Clean the text
cleaned_text = clean_text(text)
```

Description:

This function performs several cleaning tasks:

- Replaces certain special characters (such as /, @, |) with spaces.
- Converts all text to lowercase to make the analysis case-insensitive.
- Removes all numbers.
- Removes punctuation.
- Strips extra spaces, making the text more uniform.

For eg. If the raw text contained "Freedom / Justice | @Equality", after cleaning, it would become "freedom justice equality"

d. Removing Stopwords

```
# Remove stopwords
stop_words = set(stopwords.words('english'))
cleaned_text = ' '.join([word for word in cleaned_text.split() if word not in stop_words])
```

Description:

The code removes common English stopwords (e.g., "the," "and," "is") using the NLTK library. This helps focus on more meaningful words by removing those that do not carry much meaning on their own.

4. Creating a Document-Term Matrix (DTM)

```
# Convert text to a document-term matrix
vectorizer = CountVectorizer()
dtm = vectorizer.fit_transform([cleaned_text])
dtm_matrix = dtm.toarray()
```

Description:

- The CountVectorizer from sklearn is used to create a document-term matrix, where each unique word in the cleaned text becomes a feature (column) and its count is the value (the number of times the word appears).
- dtm stores the matrix where rows represent documents (in this case, one document, the speech) and columns represent the words (terms).
- dtm_matrix is a matrix representation of the word counts.

5. Extracting Word Frequencies

```
# Get the frequency of each word
word_freq = dict(zip(vectorizer.get_feature_names_out(), dtm_matrix[0]))
sorted_word_freq = dict(sorted(word_freq.items(), key=lambda item: item[1], r
reverse=True))
```

Description:

- word_freq creates a dictionary where keys are words and values are their counts in the text.
- sorted_word_freq sorts the dictionary in descending order of word frequency.

6. Displaying the Top 10 Frequent Words

```
# Create a dataframe to show top 10 frequent words
import pandas as pd
df = pd.DataFrame(list(sorted_word_freq.items()), columns=['word', 'freq'])
print(df.head(10))
```

	word	freq
0	freedom	13
1	ring	12
2	day	11
3	dream	11
4	let	11
5	every	9
6	able	8
7	one	8
8	together	7
9	faith	4

Description:

The word frequencies are transformed into a Pandas DataFrame for easy viewing, and the top 10 frequent words are displayed.

7. Generating a Word Cloud

```
# Generate and plot word cloud
wordcloud = WordCloud(width=800, height=400, background_color='white', max_words=200, random_state=1234, colormap='Dark2').generate_from_frequencies(sorted_word_freq)
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```

Description:

- The WordCloud object generates a word cloud from the sorted_word_freq dictionary.
- It plots the word cloud using matplotlib, where more frequent words appear larger.

8. Finding Frequent Terms and Associations

```
# Find terms with a frequency of 4 or higher
frequent_terms = {k: v for k, v in sorted_word_freq.items() if v >= 4}
print(f"Frequent terms with frequency >= 4: {frequent_terms}")

# Find words associated with "freedom"
# Calculate the correlation of "freedom" with other terms (a basic example, not direct term association)
if 'freedom' in word_freq:
    print(f"'freedom' appears {word_freq['freedom']} times in the text.")
else:
    print("'freedom' is not present in the text.")
```

Description:

The code filters and displays the words that appear with a frequency of 4 or higher.

The resulting word cloud and frequency counts provide a clear visual and quantitative representation of the most important themes in the speech, such as "freedom," "dream," and "ring." These reflect the key messages of hope, equality, and action, aligning closely with Martin Luther King Jr.'s powerful rhetoric in the "I Have a Dream" speech.

9. WordCloud Generated

The word cloud generated is based on Martin Luther King Jr.'s "I Have a Dream" speech. The larger words, such as "freedom," "ring," "dream," "day," "let," "together," and "faith," indicate the words with the highest frequency in the text.



Description:

a. Interpretation of the Word Cloud:

- **Dominant Themes:** The word cloud highlights key themes of the speech, with "freedom" being the most frequent word, which resonates with the central message of civil rights and equality. Other prominent words like "dream" and "faith" reflect the aspirational and hopeful tone of the speech.
- **Calls to Unity:** Words like "together," "one," and "brotherhood" emphasize the message of unity and solidarity across different races and backgrounds.
- **Freedom and Struggle:** Terms such as "nation," "mountain," "faith," and "let" indicate not just the dream of freedom but the ongoing struggle for justice and equality. The recurrent use of "let" and "ring" implies the call to action and the desire to spread freedom throughout the nation.
- **Emotional Appeal:** Words like "children," "hope," "faith," and "brotherhood" show an emotional appeal that invokes feelings of compassion and collective responsibility.

b. Text Mining Process:

- **Text Preprocessing:** We began by reading the text and cleaning it by removing special characters, punctuation, stopwords, and converting it to lowercase. This step allowed us to focus on the most significant words and themes in the speech.

- **Transformation:** By converting the text into a document-term matrix, we were able to identify the frequency of each word in the document, which is reflected in the word cloud.
- **Word Cloud Generation:** The word cloud visually emphasizes the most frequent words, helping to easily grasp the central themes of the text. The larger the word in the word cloud, the more frequently it appears in the speech.
- **Analysis:** The frequent terms found (e.g., "freedom," "ring," "day," "let") reveal King's focus on the struggle for civil rights and the aspiration for a future where freedom rings across the nation. The word cloud reflects the speech's message of hope, unity, and the pursuit of equality.

This visualization not only highlights the most frequent words but also helps to grasp the essence of the speech, showing how Dr. King's message is woven around core ideals like freedom, unity, and hope for a better future.

10. Tokenization

The code demonstrates four different tokenization techniques using the Natural Language Toolkit (nltk) in Python. Tokenization is the process of breaking down text into individual units, like words or sentences, for easier analysis in natural language processing tasks.

```
#Tokenisation

tokenizer1 = nltk.tokenize.WhitespaceTokenizer()
tokenizer2 = nltk.tokenize.TreebankWordTokenizer()
tokenizer3 = nltk.tokenize.WordPunctTokenizer()
tokenizer4 = nltk.tokenize.RegexpTokenizer(r'\w+')

print("Example Text: ",text)
print("-----")
print("Tokenization by whitespace:- \n",tokenizer1.tokenize(text))
print("Tokenization by words using Treebank Word Tokenizer:- \n",tokenizer2.tokenize(text))
print("Tokenization by punctuation:- \n",tokenizer3.tokenize(text))
print("Tokenization by regular expression:- ",tokenizer4.tokenize(text))
Tokenization by whitespace:-
['And', 'so', 'even', 'though', 'we', 'face', 'the', 'difficulties', 'of', ,
'a', 'dream', 'that', 'one',
...
, 'the', 'old', 'Negro', 'spiritual:', 'Free', 'at', 'last!', 'Free', 'at', 'last!', 'Thank', 'God', 'Almighty,', 'we', 'are', 'free', 'at', 'last!']
Tokenization by words using Treebank Word Tokenizer:-
```



```
['And', 'so', 'even', 'though', 'we', 'face', 'the', 'difficulties', 'of', '
today', 'and', 'tomorrow', ',', 'I', 'still', 'have', 'a',
...
', 'Negro', 'spiritual', ':', 'Free', 'at', 'last', '!', 'Free', 'at', 'last'
, '!', 'Thank', 'God', 'Almighty', ',', 'we', 'are', 'free', 'at', 'last', '!'
']
```

Tokenization by punctuation:-

```
['And', 'so', 'even', 'though', 'we', 'face', 'the', 'difficulties', 'of', '
today', 'and', 'tomorrow', ',', 'I', 'still', 'have', 'a', 'dream', '.', 'It'
...
gro', 'spiritual', ':', 'Free', 'at', 'last', '!', 'Free', 'at', 'last', '!',
'Thank', 'God', 'Almighty', ',', 'we', 'are', 'free', 'at', 'last', '!']
```

Tokenization by regular expression:- ['And', 'so', 'even', 'though', 'we', '
face', 'the', 'difficulties', 'of', 'today', 'and', 'tomorrow', 'I', 'still',
...
, 'the', 'old', 'Negro', 'spiritual', 'Free', 'at', 'last', 'Free', 'at', 'la
st', 'Thank', 'God', 'Almighty', 'we', 'are', 'free', 'at', 'last']

Description:

- **Whitespace Tokenization:** This method splits the text based on spaces. It's useful when we need the tokens as they appear in the text without considering punctuation or special characters. The output includes words separated by any form of whitespace.
- **Treebank Word Tokenization:** This method tokenizes text based on linguistic rules from the Penn Treebank corpus. It handles contractions like "don't" as ["do", "n't"] and splits punctuation from words more intelligently than whitespace-based methods. It provides more granular tokenization for certain language structures.
- **Punctuation Tokenization:** This tokenizer splits the text into tokens, dividing words and punctuation into separate tokens. It creates detailed tokens by treating punctuation marks as individual tokens, making it useful for analyzing text where punctuation holds significant meaning.
- **Regular Expression Tokenization :** This method uses a regular expression pattern (\w+) to tokenize the text. Here, \w+ matches sequences of word characters (letters and digits) while ignoring punctuation and whitespace. This method gives us only words and numbers, removing any special characters.

11. Stemming and Lemmetization

This code demonstrates the difference between **stemming** and **lemmatization**, two important techniques in natural language processing (NLP) for text normalization.

a. Stemming

```
# Stemming and Lemmatization examples

tokenizer = nltk.tokenize.TreebankWordTokenizer()
tokens = tokenizer.tokenize(text)

# Stemmer
stemmer = nltk.stem.PorterStemmer()
print("Stemming the sentence: ", " ".join(stemmer.stem(token) for token in tokens))
Stemming the sentence:  and so even though we face the difficulti of today an
d tomorrow , i still have a dream. it is a dream deepli root in the american
dream. i have a dream that one day thi nation will rise up and live out the t
rue mean of it creed : we hold these truth to be self-evid , that all men are
creat equal.
...
children , black men and white men , jew and gentil , protest and cathol , wi
ll be abl to join hand and sing in the word of the old negro spiritu : free a
t last ! free at last ! thank god almighti , we are free at last !
```

Description:

Stemming: Stemming reduces words to their root or base form by chopping off affixes like "ing," "ly," "ed," etc., without considering the context or part of speech. The PorterStemmer in nltk is a commonly used algorithm for this purpose. For example, it reduces words like "running," "runs," and "runner" to "run." However, stemming can sometimes result in non-dictionary words (e.g., "better" may become "bet"). In the code, the stemmer.stem() function is applied to each tokenized word to produce its stemmed form.

b. Lemmatization

```
import nltk
from nltk.corpus import stopwords
nltk.download('wordnet')

# Lemmatizer
lemmatizer=nltk.stem.WordNetLemmatizer()
print("Lemmatizing the sentence: ", " ".join(lemmatizer.lemmatize(token) for token in tokens))
[nltk_data] Downloading package wordnet to
[nltk_data] /Users/shraddha/nltk_data...
Lemmatizing the sentence:  And so even though we face the difficulty of today
and tomorrow , I still have a dream. It is a dream deeply rooted in the Ameri
can dream. I have a dream that one day this nation will rise up and live out
```

the true meaning of it creed : We hold these truth to be self-evident , that all men are created equal.

...

men , Jews and Gentiles , Protestants and Catholics , will be able to join hand and sing in the word of the old Negro spiritual : Free at last ! Free at last ! Thank God Almighty , we are free at last !

Description:

Lemmatization: Lemmatization goes a step further by considering the context and reducing words to their dictionary form, or "lemma." It takes into account the part of speech and produces valid words (e.g., "better" stays as "better" instead of "bet"). The WordNetLemmatizer is used here to lemmatize each token, ensuring that words are reduced to their base form while retaining meaning. It is more sophisticated than stemming and often yields more accurate results, though it can be slower due to the additional linguistic context required.

Both techniques are crucial for preparing textual data, allowing words to be compared in their base forms and reducing redundancy in datasets for tasks like text classification or sentiment analysis.

12. Evaluation

Based on the output generated from the analysis of Martin Luther King Jr.'s "I Have a Dream" speech, several key insights and relationships among the words in the text emerge.

a. Prominent Themes

The most frequent words in the speech are freedom (13 occurrences), ring (12), day (11), dream (11), let (11), and every (9). These words form the backbone of King's message, emphasizing the central ideas of freedom, unity (as seen in "ring" and "together"), and the pursuit of a shared dream for equality and justice.

.

A table of the top 10 frequent words allows us to quantitatively understand which words dominate the speech:

Word	Frequency
freedom	13
ring	12
day	11

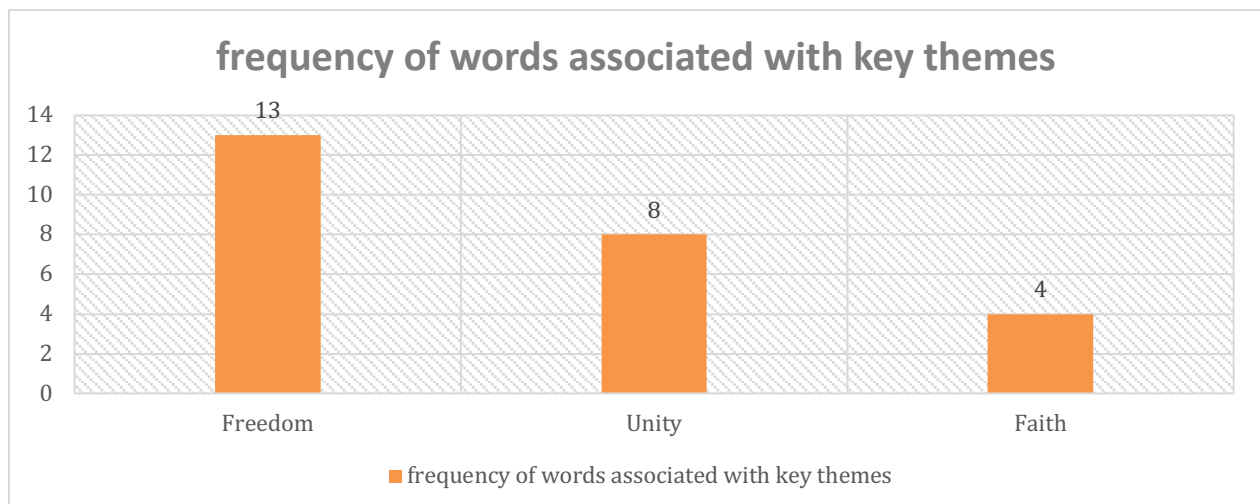
dream	11
let	11
every	9
able	8
one	8
together	8
faith	4

b. Word Cloud Analysis

The word cloud visualization shows that freedom, ring, dream, day, and let are visually more prominent due to their higher frequency. These larger words reveal that these are the primary ideas conveyed in the speech, while smaller words like faith, hope, nation, and together contribute secondary but significant themes.

c. Thematic Analysis

- **Unity and Collective Action:** Words like let, together, and every emphasize the collective responsibility to bring about change. The use of we and our further strengthens the idea that the fight for freedom is shared by all Americans.
- **Faith and Hope:** Words like faith (4 mentions) and hope reflect a belief in the possibility of change despite adversity. These words communicate optimism, reinforcing that King's dream is not just aspirational but achievable.



Description:

The word cloud and frequent terms give us a clear picture of how Martin Luther King Jr. articulated his vision — by repeatedly focusing on freedom, unity, and action, and emphasizing the importance of collective effort in achieving racial equality.

13. Interpretations

The analysis of Martin Luther King Jr.'s "I Have a Dream" speech reveals several key insights about the recurring themes and how they were communicated. For the **data owner**, these findings shed light on the speech's effectiveness in delivering a powerful message about **freedom, unity, and collective action**. By breaking down the word frequency and associations, we can understand how these central themes resonate with the audience.

a. Main Themes: Freedom, Unity, and Action

- **Freedom** is the most dominant theme in the speech, as shown by the 13 mentions of the word. This indicates that the primary focus of King's message is the pursuit of liberty and equal rights for Black Americans.
- **Unity** and **collective action** are also crucial, as seen from the frequent use of words like **together, let, and every**. King appeals to all people, emphasizing that achieving freedom and equality requires the involvement of every individual, regardless of background.

b. Emphasis on Inclusivity and Hope

- Words like every, one, and faith illustrate an inclusive message aimed at uniting people across racial and social divides. The recurrence of faith and hope shows that the speech carries a message of optimism, suggesting that the audience should believe in the possibility of change.

14. Recommendations:

Based on the findings of this analysis, I would recommend the following actions to the data owner

a. Leverage Key Themes in Future Messaging

Freedom is the cornerstone of King's message. Future communications or educational programs that build on this speech should continue emphasizing the importance of freedom and equality as

core values. The word **freedom** appeared 13 times, making it the most frequent word in the speech. This should guide the framing of future advocacy, continuing to push for these ideals.

b. Call to Action through Collective Unity:

Emphasize **unity** and **collective effort**. King's repeated use of words like **together**, **let**, and **every** suggests that change is not an individual pursuit but a collective mission. Words related to unity (e.g., **let**, **together**, **one**) appeared frequently, reinforcing the idea that achieving civil rights requires a group effort.

c. Highlight Messages of Hope and Inclusivity

King's message balances urgency with optimism. For any educational or advocacy campaign, it would be useful to focus on this balance by highlighting how **faith** and **hope** play roles in fostering long-term change. Words like **faith** (4 times) and **hope** are crucial elements of King's speech. Educational materials can reflect these ideas to maintain a positive outlook while addressing tough issues.

15. Explicit Variables to Incorporate in Further Analysis:

For further analysis and to gain deeper insights, the following variables could be incorporated:

a. Sentiment Analysis:

This would quantify the tone of the speech and determine how much of it is positive (hopeful) versus negative (acknowledgment of struggles). Sentiment analysis could complement the frequency analysis to reveal the emotional appeal in the speech.

b. Comparative Analysis of Other Speeches:

Analyzing other speeches by Martin Luther King Jr. or other leaders in the civil rights movement could reveal whether certain words and themes recur or if they evolve over time. This would provide context on how King's messaging adapted to different moments in the movement.

c. Time and Context Variables:

In analyzing the speech, it would be useful to incorporate historical context. Variables such as the **year** of the speech, current **political climate**, and **social movements** could be introduced to assess how external factors influenced the language used in the speech.

d. Audience Analysis:

Variables representing the **audience's demographics** (e.g., race, education level, location) could help evaluate how different groups respond to the themes in King's speech. This could be tied to how persuasive or impactful certain words or messages were to different audiences.

16. Conclusion:

The speech's recurring focus on **freedom**, **unity**, and **action** makes it a timeless call for equality. This analysis offers practical insights into how to maintain the relevance and power of these messages in contemporary advocacy efforts. By understanding how language is used to convey both vision and urgency, educators, advocates, and policymakers can create strategies that resonate with today's audiences, fostering a collective drive towards meaningful change.

17. References

- a. IBM. (n.d.). **Stemming and lemmatization: What's the difference?** IBM. Retrieved October 20, 2024, from <https://www.ibm.com/topics/stemming-lemmatization>
- b. Tutorialspoint. (n.d.). **Natural language processing tutorial**. Tutorialspoint. Retrieved October 20, 2024, from https://www.tutorialspoint.com/natural_language_processing/index.htm
- c. GeeksforGeeks. (2023). **Tokenization in natural language processing (NLP)**. GeeksforGeeks. Retrieved October 20, 2024, from <https://www.geeksforgeeks.org/tokenization-in-natural-language-processing-nlp>
- d. U.S. Embassy & Consulate in the Republic of Korea. (n.d.). **Martin Luther King Jr.'s "I Have a Dream" speech (1963)**. U.S. Embassy & Consulate in the Republic of Korea. Retrieved October 20, 2024, from <https://kr.usembassy.gov/martin-luther-king-jr-dream-speech-1963>